

Reporte de Analisis Inicial

Dr. Bike Sydney

Repositorio: `github.com/Peredodiego2026/Dr.Bike-Sydney` **Stack:** Vercel + Supabase + Stripe + Twilio + Resend + Anthropic (Claude) **Estado:** Pre-lanzamiento **Fecha del analisis:** Junio 2026

Resumen Ejecutivo

Metrica	Valor
Archivos totales	47 en raiz
APIs serverless	13 endpoints
Version del proyecto	1.0.0
Dependencias	6 (Anthropic, Supabase, Resend, Stripe, Twilio, Web Push)
Tamano aprox.	~30 MB (con node_modules)

Hallazgos: 4 criticos, 4 medios, 10 bajos identificados.

Arquitectura General

```
Dr. Bike Sydney (PWA Serverless)
|
+-- Vercel (Hosting + Cron)
|   +-- Cron cada 15min: /api/send-2h-reminders
|   +-- Ruteo SPA: todo a index.html
|
+-- Supabase (Base de datos + Auth + Realtime)
|   +-- Tablas: bookings, profiles, services, vans, parts_inventory
|   +-- RLS: NO habilitado (riesgo alto)
|
+-- Stripe (Pagos)
|   +-- Checkout para membresias
|   +-- Webhooks para eventos de suscripcion
|
+-- Twilio (SMS + WhatsApp)
|   +-- Recordatorios, confirmaciones, notificaciones
|
+-- Resend (Email transaccional)
|   +-- Confirmaciones, facturas, recordatorios
|
+-- Anthropic (AI Chat + Diagnostico)
    +-- /api/chat.js - Asistente virtual Haiku 4.5
    +-- /api/diagnose-bike.js - Diagnostico con imagenes
```

2 Sistemas de Diseno Conviviendo

```
index.html (claro)                landing.html (oscuro)
+-----+                +-----+
| Fondo: blanco      |        | Fondo: oscuro      |
| Ac: azul #2563eb   |        | Ac: azul #0A58CA   |
| CSS inline 49001n  |        | CSS propio 30001n   |
+-----+                +-----+
      \                    /
      \                    /
index-redesign.html (navy oscuro 48001n)
Fondo: #0a0e1a, CSS inline
```

Problema: 3 sistemas de diseno distintos para la misma marca. Sin design tokens compartidos.

Hallazgos de Seguridad

ID	Componente	Problema	Riesgo
S-01	index.html:55	Google Maps API key hardcodeada: AIzaSyC0rbGMevQfxPNRnes2rRZktUA208hP6nY	CRITICO
S-02	admin.html:3440	Auth solo client-side, sin RLS en Supabase. Password en JS.	CRITICO
S-03	send-email.js / send-invoice.js	XSS en templates HTML: datos de usuario interpolados sin escape	CRITICO
S-04	mobile_latest.html:2845	Eruda debug console expuesta en produccion	ALTO
S-05	_security.js	Rate limiting en memoria (por instancia), no compartido	MEDIO
S-06	_security.js:verifyInternalAuth	Auth basada en headers Origin/Referer (spoofeables)	MEDIO
S-07	create-subscription.js:51	Restringe solo a card, bloquea Apple Pay / Google Pay	MEDIO
S-08	scripts/add-bookings-rls.sql	Script RLS comentado: "do NOT execute yet"	MEDIO

Hallazgos de Calidad deCodigo

ID	Archivo	Problema	Impacto
C-01	index.html	Monolito de 4,898 lineas (HTML + CSS + JS)	Mantenimiento

C-02	admin.html	Monolito de 3,497 lineas	Mantenimiento
C-03	mobile.html, v2, v3, latest	4 versiones de la misma app	Confusion
C-04	index.html VS index-redesign.html	95% identicos (4800 vs 4900 lineas)	Duplicacion
C-05	applepay.html VS applepay-test.html	100% identicos	Duplicacion
C-06	admin.html.bak	Backup en raiz de deployment	Riesgo
C-07	payments.html, notifications.html	Mockups guardados desde navegador con rutas locales rotas	Dead code
C-08	send-sms.js / send-whatsapp.js	Funcion normalizeAUPhone() duplicada en 2 archivos	Repetition

Hallazgos de Bugs

ID	Archivo	Bug	Impacto
B-01	stripe-webhook.js:233	invoice.payment_failed USA membership en vez de membership_status	Funcional
B-02	send-email.js	Template referral_success con variables friendName y referralCount fuera de scope	Funcional
B-03	send-whatsapp.js	Numero WhatsApp leido de tabla van_zones con hack van_number=0, suburb='___whatsapp__'	Fragilidad
B-04	package.json	@anthropic-ai/sdk listado pero todos los endpoints usan fetch directo	Dependencia muerta

Distribucion de Archivos por Tipo

HTML	#####	12 archivos
API	#####	13 endpoints
JS	####	2 scripts + sw.js
SQL	###	4 scripts
CSS	.	0 (todo inline en HTML)
PNG	#####	8 iconos
SVG	#####	11 iconos
JSON	####	6 archivos config
BAT	.	0

Notable: 0 archivos CSS separados. Todo el CSS esta inline en archivos HTML.

Archivos por Tamanio

Archivo	Lineas	Tamano	Contenido
index.html	4,898	261 KB	App principal cliente
index-redesign.html	4,803	257 KB	Rediseno (duplicado 95%)
admin.html	3,497	212 KB	Panel administrador
mobile_latest.html	2,853	~150 KB	App mobile cliente
mobile_v2.html	1,817	~100 KB	Version mobile (obsoleta)
mobile_v3.html	1,817	~100 KB	Version mobile (obsoleta)
admin.html.bak	1,551	~80 KB	Backup admin
mechanic.html	1,152	~60 KB	App mecanico
landing.html	3,026	106 KB	Landing SEO
applepay.html	168	~8 KB	Test Apple Pay
applepay-test.html	168	~8 KB	Duplicado exacto

Dependencias

```
@anthropic-ai/sdk    - NO USADO (fetch directo a API)
@supabase/supabase-js - Cliente Supabase
resend                - Email transaccional
stripe                - Pagos
twilio                - SMS/WhatsApp
web-push              - Notificaciones push
```

Vulnerabilidades Criticas (Deep Dive)

S-01: API Key de Google Maps Expuesta

Archivo: index.html (linea 55)

La key AIzaSyC0rbGMevQfxPNRnes2rRZktUA208hP6nY esta hardcodeda en el HTML. Cualquier persona que abra el inspector del navegador puede extraerla y usarla en otros sitios.

Riesgo: Uso no autorizado, costos inesperados en Google Cloud.

Solucion: Restringir la key por HTTP referrer en la consola de Google Cloud.

S-02: Admin Sin Autenticacion Real

Archivo: admin.html:3440

El admin se protege con un password en JavaScript del lado del cliente. Cualquiera que lea el codigo fuente puede ver (y modificar) este mecanismo. Supabase no tiene RLS habilitado.

Riesgo: Acceso no autorizado a todos los datos de la aplicacion.

Solucion: Implementar Supabase Auth + RLS.

S-03: XSS en Templates de Email

Archivos: `api/send-email.js` , `api/send-invoice.js`

Datos del usuario (nombre, direccion, notas) se interpolan directamente en HTML sin escapar. Si un usuario registra un nombre como `<script>...</script>` , se ejecuta en el correo.

Riesgo: Robo de sesiones de email, phishing interno.

Solucion: Usar `sanitize()` de `_security.js` en todas las interpolaciones.

S-04: Consola Eruda en Produccion

Archivo: `mobile_latest.html:2845`

Eruda es una consola de desarrollo completa (DOM, Network, Console, Resources) incluida en la version que se sirve a usuarios. Cualquier usuario puede inspeccionar peticiones a Supabase y Stripe.

Riesgo: Fuga de datos de API, inspeccion de peticiones.

Solucion: Eliminar el script de Eruda en produccion.

Metricas del Codebase

Metrica	Valor	Evaluacion
Archivos totales	~47	Normal
Lineas de HTML	~24,000	Alto (mucho inline)
Lineas de JS (API)	~2,500	Moderado
Lineas de CSS	~15,000 (inline)	Muy alto (todo inline)
API endpoints	13	Normal
Dependencias npm	6	Bajo
Archivos duplicados	3 pares	Malo
RLS habilitado	NO	Critico
Codigo muerto	~5,000 lineas estimado	Significativo

Prioritizacion de Acciones

Prioridad	Accion	Sesion del plan
ALTA	Eliminar Eruda de produccion	Saneamiento S1

ALTA	Sanitizar templates de email contra XSS	Saneamiento S1
ALTA	Documentar restriccion de Google Maps key	Saneamiento S1
ALTA	Habilitar RLS en Supabase (bookings + profiles)	Saneamiento S2
ALTA	Implementar auth server-side para admin	Saneamiento S2
MEDIA	Eliminar codigo muerto (4x mobile, duplicados, .bak)	Saneamiento S3
MEDIA	Corregir bugs: stripe-webhook, send-email	Saneamiento S4
MEDIA	Modularizar frontend (4900 -> ~800 lineas)	Saneamiento S5
BAJA	Rate limiting con Redis	Saneamiento S6
BAJA	Service worker: limpieza selectiva	Saneamiento S6
BAJA	Implementar Apple Pay / Google Pay	Saneamiento S6

Resumen de Plan de Saneamiento

```

Sesion 1: Seguridad critica ..... ~30 min
Sesion 2: Auth + RLS ..... ~45 min
Sesion 3: Dead code cleanup ..... ~20 min
Sesion 4: Bug fixes ..... ~30 min
Sesion 5: Modularizacion frontend ..... ~60 min
Sesion 6: Produccion readiness ..... ~40 min
-----
Total estimado: ..... ~3h 45min

```

Resumen de Plan de Rediseno UI

```

Sesion 1: Design system + SPA shell ..... ~45 min
Sesion 2: Home + Service Type ..... ~45 min
Sesion 3: Date/Time + Summary ..... ~40 min
Sesion 4: Payment + Tracking ..... ~50 min
Sesion 5: Review + Auth + Bookings ..... ~40 min
Sesion 6: PWA + Unificacion + Polish .... ~40 min
-----
Total estimado: ..... ~4h 40min

```

Recomendaciones

1. **Ejecutar saneamiento antes que rediseño** -- la base limpia evita propagar problemas
2. **Habilitar RLS inmediatamente** -- es el riesgo mas grave
3. **Unificar a un solo design system** antes de agregar nuevas pantallas
4. **Eliminar codigo muerto temprano** -- reduce confusion y riesgo de tocar archivos equivocados
5. **No agregar nuevas features** hasta completar ambos planes
6. **Configurar Google Maps API key con restriccion HTTP referrer** en la consola de Google Cloud

Fin del reporte. Documento generado a partir del analisis del repositorio github.com/Peredodiego2026/Dr.Bike-Sydney .